

OCR Level 3 Advanced GCE in Computer Science (H446)

Specification

Version 1: First Assessment 2017

This draft qualification has not yet been accredited by Ofqual. It is published to enable teachers to have early sight of our proposed approach to A Level in Computer Science. Further changes may be required and no assurance can be given at this time that the proposed qualification will be made available in its current form, or that it will be accredited in time for first teaching in 2015 and first award in 2017.

Draft

Contents

1	Why choose an OCR A Level in Computer Science?	3
1a.	Why choose an OCR qualification?	3
1b.	Why choose an OCR A Level in Computer Science?	4
1c.	What are the key features of this specification?	5
1d.	How do I find out more information?	5
2	The specification overview	6
2a.	Overview of A Level in Computer Science (H446)	6
2b.	Content of A Level in Computer Science (H446)	7
2c.	Content of Computer systems (Component 01)	8
2c.	Content of Algorithms and programming (Component 02)	12
2c.	Content of non exam assessment Programming project (Component 03 or 04)	14
2d.	Prior learning and progression	16
3	Assessment of OCR A Level in Computer Science	17
3a.	Forms of assessment	17
3b.	Assessment availability	18
3c.	Assessment objectives (AO)	18
3d.	Assessment of extended responses	19
3e.	Non exam assessment	19
3f.	Synoptic assessment	26
3g.	Calculating qualification results	26
4	Admin: what you need to know	27
4a.	Pre-assessment	27
4b.	Accessibility and special consideration	28
4c.	External assessment arrangements	28
4d.	Non exam assessment	28
4e.	Results and certificates	29
4f.	Post-results services	30
4g.	Malpractice	30
5	Appendices	31
5a.	Grade descriptors	31
5b.	Overlap with other qualifications	31
5c.	Mathematical skills	31
5d.	Languages and logic	31

1 Why choose an OCR A Level in Computer Science?

1a. Why choose an OCR qualification?

Choose OCR and you've got the reassurance that you're working with one of the UK's leading exam board. Our new A Level in Computer Science course has been developed in consultation with teachers, employers and Higher Education to provide students with a qualification that's relevant to them and meets their needs.

We're part of the Cambridge Assessment Group, Europe's largest assessment agency and a department of the University of Cambridge. Cambridge Assessment plays a leading role in developing and delivering assessments throughout the world, operating in over 150 countries.

We work with a range of education providers, including schools, colleges, workplaces and other institutions in both the public and private sectors. Over 13,000 centres choose our A levels, GCSEs and vocational qualifications including Cambridge Nationals, Cambridge Technicals and Cambridge Progression.

Our Specifications

We believe in developing specifications that help you bring the subject to life and inspire your students to achieve more.

We've created teacher-friendly specifications based on extensive research and engagement with the teaching community. They're designed to be straightforward and accessible so that you can tailor the delivery of the course to suit your needs. We aim to encourage learners to become responsible for their own learning, confident in discussing ideas, innovative and engaged.

We provide a range of support services designed to help you at every stage, from preparation through to the delivery of our specifications. This includes:

A wide range of high-quality creative resources including:

- Delivery Guides
- Transition Guides
- Topic Exploration Packs
- Lesson Elements
- ...and much more.
- Access to subject specialists to support you through the transition and throughout the lifetimes of the specifications.
- CPD/Training for teachers including face-to-face events to introduce the qualifications and prepare you for first teaching.
- Active Results – our free results analysis service to help you review the performance of individual students or whole schools.
- ExamCreator – our new online past papers service that enables you to build your own test papers from past OCR exam questions.

All A level qualifications offered by OCR are accredited by Ofqual, the Regulator for qualifications offered in England.

1b. Why choose an OCR A Level in Computer Science?

'At its heart lies the notion of computational thinking: a mode of thought that goes well beyond software and hardware, and that provides a framework within which to reason about systems and problems.'

CAS Computer Science a Curriculum for Schools

This specification has been developed by the team that created the first GCSE Computing qualification in the UK. The experience of the past three years of assessment has clearly demonstrated that OCR has the knowledge and skills to develop reliable and valid qualifications in this area of study.

OCR Computer Science will above all else be relevant to the modern and changing world of Computing. It enables teachers to be able to tailor the qualification to meet the needs of their learners in their centre and has an open source ethos allowing any programming language that meets the needs of the course to be used.

Computer Science is a practical subject where learners can apply the academic principles learned in the classroom to real world systems.

It is an intensely creative subject that combines invention and excitement, and can look at the natural world through a digital prism. OCR A Level in Computer Science will value computational thinking, helping learners to develop the skills to solve problems, design systems and understand the power and limits of human and machine intelligence.

Learners will develop an ability to analyse, critically evaluate and make decisions. The project approach is a vital component of 'post-school' life and is of particular relevance to Further Education, Higher Education and the workplace. Each learner is able to tailor their project to fit their individual needs, choices and aspirations. OCR offers a rigorous assessment structure that ensures the integrity of the project.

Aims and learning outcomes

The aims of this qualification are to enable learners to:

- develop an understanding of and ability to apply the fundamental principles and concepts of computer science including; abstraction, decomposition, logic, algorithms and data representation
- develop the ability to analyse problems in computational terms through practical experience of solving such problems including writing programs to do so
- develop the capacity for thinking creatively, innovatively, analytically, logically and critically
- develop the capacity to see relationships between different aspects of computer science
- develop mathematical skills
- develop the ability to articulate the individual (moral), social (ethical), legal and cultural opportunities and risks of digital technology.

1c. What are the key features of this specification?

The OCR A Level in Computer Science will encourage learners to be inspired, motivated and challenged by following a broad, coherent, practical, satisfying and worthwhile course of study. It will provide insight into, and experience of how computer science works, stimulating learners' curiosity and encouraging them to engage with computer science in their everyday lives and to make informed choices about further study or career choices.

The key features of this specification encourage:

- emphasis on problem solving using computers
- emphasis on computer programming and algorithms

- emphasis on the mathematical skills used to express computational laws and processes, e.g. Boolean algebra/logic and comparison of the complexity of algorithms
- less emphasis on ICT.

Centres and learners have the opportunity to:

- produce a slimmed down programming project which is more refined and more focussed on coding
- choose the project title and problem to be solved
- choose any suitable programming language
- include agile methods.

1d. How do I find out more information?

If you are already using OCR specifications you can contact us at: www.ocr.org.uk

If you are not already a registered OCR centre then you can find out more information on the benefits of becoming one at: www.ocr.org.uk

If you are not yet an approved centre and would like to become one go to: www.ocr.org.uk

Find out more?

Ask a subject expert:

Email: OCR.CompSciAndHSC@ocr.org.uk

Customer Contact Centre: 01223 553998

Teacher support: www.ocr.org.uk

News: www.ocr.org.uk

2 The specification overview

2a. Overview of A Level in Computer Science (H446)

Learners must take three components (01, 02 and 03 or 01, 02 and 04).

Content Overview	Assessment Overview	
• Characteristics of contemporary processors • Software and development • Exchanging data • Data types structures and algorithms • Legal, moral and ethical issues • Elements of computational thinking • Problem solving and programming • Algorithms to solve problems and standard algorithms • Analysis of the problem • Design of the solution • Developing the solution • Evaluation	Computer systems (01) 140 marks 2 hours and 30 minutes written paper	40% of total A level
	Algorithms and programming (02*) 140 marks 2 hours and 30 minutes written paper	40% of total A level
	Programming project (03* or 04*) 70 marks Non-exam assessment	20% of total A level

* Indicates synoptic assessment

Learners who are re-taking the qualification may carry forward their mark for the non exam assessment component (04).

2b. Content of A Level in Computer Science (H446)

The content of this A Level in Computer Science is divided into three components:

- Computer systems component (H446/01) contains the majority of the content of the specification and is assessed in a written paper recalling knowledge and understanding
- Algorithms and programming component (H446/02) relates principally to problem solving skills needed by learners to apply the knowledge and understanding encountered in component 01
- Programming project component (H446/03 or H446/04) is a practical portfolio based assessment with a task that is chosen by the teacher or learner and is produced in an appropriate programming language of the learner's or teacher's choice.

The specification has been designed to be co-teachable with the stand alone AS Level in Computer Science.

The mathematical skills are embedded throughout the content of the three components. They will be assessed in the written papers and through the non examined assessment where appropriate. The quality of extended responses are assessed in the written papers where indicated by an asterisk. It is assessed by levels of response style mark schemes and in the Evaluation section of the Programming project component.

Draft

2c. Content of Computer systems (Component 01)

This component will introduce learners to the internal workings of the Central Processing Unit (CPU), the exchanging of data and also looks at software development, data types and legal and ethical issues. It is expected that learners will draw on this underpinning content when studying computational

thinking, developing programming techniques and devising their own programming approach in the Programming project component (03 or 04). Learners will be expected to apply the criteria below in different contexts including current and future uses of the technologies.

1.1 The characteristics of contemporary processors, input, output and storage devices	
Components of a computer and their uses	
1.1.1 Structure and function of the processor	a) The Arithmetic and Logic Unit; ALU, Control Unit and Registers (Program Counter; PC, Accumulator; ACC, Memory Address Register; MAR, Memory Data Register; MDR, Current Instruction Register; CIR): How this relates to assembly language programs. b) The Fetch-Decode-Execute Cycle. c) The use of pipelining in a processor to improve efficiency.
1.1.2 Types of processor	a) The differences between and uses of CISC and RISC processors. b) GPUs and their uses (including those not related to graphics). c) Multicore and Parallel systems.
1.1.3 Input, output and storage	a) How different input output and storage devices can be applied to the solution of different problems. b) The uses of magnetic, flash and optical storage devices. c) RAM and ROM. d) Virtual storage.

1.2 Software and software development	
Types of software and the different methodologies used to develop software	
1.2.1 Systems Software	a) The function and purpose of operating systems. b) Memory Management (paging, segmentation and virtual memory). c) Interrupts. d) Scheduling: Round Robin, First come first served, Multi-level feedback queues, shortest job first and shortest remaining time. e) Distributed, Embedded, Multi-Tasking, Multi-User and Real Time operating systems. f) BIOS. g) Device drivers. h) Virtual machines.
1.2.2 Applications Generation	a) The nature of applications. b) Utilities. c) Open source vs Closed source. d) Translators: Interpreters, compilers and assemblers. e) Stages of compilation (Lexical Analysis, Syntax Analysis, Code Generation and Optimisation). f) Linkers and loaders.
1.2.3 Software Development	a) Understand the waterfall lifecycle, agile methodologies, extreme programming, the spiral model and rapid application development. b) Writing and following algorithms. c) The relative merits and drawbacks of different methodologies and when they might be used.
1.2.4 Types of Programming Language	a) Procedural languages. b) Assembly language (including following and writing simple programs with the Little Man Computer instruction set). c) Modes of addressing memory (immediate, direct, indirect and indexed). d) Object-oriented languages (using Java/C++ style pseudocode) with an understanding of classes, objects, methods, attributes, inheritance, encapsulation and polymorphism.

1.3 Exchanging data	
How data is exchanged between different systems	
1.3.1 Compression, Encryption and Hashing	a) Lossy vs Lossless compression. b) Run Length Encoding and dictionary coding for lossless compression. c) Symmetric and asymmetric encryption. d) Different uses of hashing.
1.3.2 Databases	a) Relational database, flat file, primary key, foreign key, secondary key, normalisation and indexing. b) Methods of capturing, selecting, managing and exchanging data. c) Normalisation to 3NF. d) SQL - Interpret and modify (list of key words). e) Referential Integrity. f) Transaction processing, ACID (Atomicity, Consistency, Isolation, Durability), record locking and redundancy.
1.3.3 Networks	a) The TCP/IP Stack. b) Protocol layering. c) LANs and WANs. d) Packet and circuit switching. e) Protocols. f) Client-server and Peer to peer.
1.3.4 Web Technologies	a) HTML, CSS and JavaScript. b) Search engine indexing. c) PageRank Algorithm. d) Server and client side processing.

1.4 Data types, data structures and algorithms	
How data is represented and stored within different structures. Different algorithms that can be applied to these structures	
1.4.1 Data Types	a) Represent positive integers in binary. b) Use of Sign and Magnitude and Two's Complement to represent negative numbers in binary. c) Addition and subtraction of binary integers. d) Represent positive integers in hexadecimal. e) Representation and normalisation of floating point numbers in binary. f) Floating point arithmetic, positive and negative numbers, addition and subtraction. g) Bitwise manipulation and masks: shifts, combining with AND, OR, and XOR. h) How character sets (ASCII and UNICODE) are used to represent text.
1.4.2 Data Structures	a) Arrays (of up to 3 dimensions). b) The following structures to store data: linked-list, graph (directed and undirected), stack, queue, tree, binary search tree, hash table. c) How to create, traverse, add data to and remove data from the data structures mentioned above. <i>(NB this can be either using arrays and procedural programming or an object-oriented approach).</i>
1.4.3 Boolean Algebra	a) Define problems using Boolean logic. b) Use the following rules to derive or simplify statements in Boolean algebra: De Morgan's Laws, distribution, association, commutation, double negation.
1.5 Legal, moral and ethical Issues	
Laws surrounding the use of computers and ethical issues that can or may in the future arise as from the use of computers	
1.5.1 Computing related laws	a) Data Protection Act. b) Computer Misuse Act. c) Copyright and Patents Act. d) Regulation of Investigatory Powers Act.
1.5.2 Moral and ethical Issues	a) Arrays (of up to 3 dimensions). b) The following structures to store data: linked-list, graph (directed and undirected), stack, queue, tree, binary search tree, hash table. c) How to create, traverse, add data to and remove data from the data structures mentioned above. <i>(NB this can be either using arrays and procedural programming or an object-oriented approach).</i>

2c. Content of Algorithms and programming (Component 02)

This component will incorporate and build on the knowledge and understanding gained in the Computer systems component (01).

In addition, learners should:

- understand what is meant by computational thinking
- understand the benefits of applying computational thinking to solving a wide variety of problems
- understand the principles of solving problems by computational methods
- be able to analyse a problem by identifying its component parts.

2.1 Elements of computational thinking	
Understand what is meant by computational thinking	
2.1.1 Thinking abstractly	a) The nature of abstraction. b) The need for abstraction. c) The differences between an abstraction and reality. d) Devise an abstract model for a variety of situations.
2.1.2 Thinking ahead	a) Identify the inputs and outputs for a given situation. b) Determine the preconditions for devising a solution to a problem. c) The nature, benefits and drawbacks of caching. d) The need for reusable program components.
2.1.3 Thinking procedurally	a) Identify the components of a problem. b) Identify the components of a solution to a problem. c) Determine the order of the steps needed to solve a problem. d) Identify sub-procedures necessary to solve a problem.
2.1.4 Thinking logically	a) Identify the points in a solution where a decision has to be taken. b) Determine the logical conditions that affect the outcome of a decision. c) Determine how decisions affect flow through a program.
2.1.5 Thinking concurrently	a) Determine the parts of a problem that can be tackled at the same time. b) Outline the benefits and trade offs that might result from concurrent processing in a particular situation.

2.2 Problem solving and programming	
How computers can be used to solve problems and programs can be written to solve them (Learners will benefit from being able to program in a procedure/imperative language.)	
2.2.1 Programming techniques	a) Programming constructs: sequence, iteration, branching. b) Recursion, how it can be used and compares to an iterative approach. c) Global and local variables. d) Modularity, functions and procedures, parameter passing by value and by reference. e) Use of an IDE to develop/debug a program.
2.2.2 Computational methods	a) Features that make a problem solvable by computational methods. b) Problem Recognition. c) Problem Decomposition. d) Use of divide and conquer. e) Use of abstraction. f) Learners should apply their knowledge of: • backtracking • data mining • heuristics • performance modelling • pipelining • visualisation to solve problems.

2.3 Algorithms	
The use of algorithms to describe problems and standard algorithms	
2.3.1 Algorithms	a) Analysis and design of algorithms for a given situation. b) The suitability of different algorithms for a given task and data set, in terms of execution time and space. c) Comparison of the complexity of algorithms. d) Algorithms for the main data structures, (Stacks, queues, trees, linked lists, depth-first (post-order) and breadth-first traversal of trees). e) Standard algorithms (Bubble sort, insertion sort, merge sort, quick sort, Dijkstra's shortest path algorithm, A* algorithm, binary search and linear search).

2c. Content of non exam assessment Programming project (Component 03 or 04)

Learners will be expected to analyse, design, develop, test, evaluate and document a program written in a suitable programming language. The underlying approach to the project is to apply the principles of computational thinking to a practical coding problem. Learners are expected to apply appropriate principles from an agile

development approach to the project development.

While the project assessment criteria are organised into specific categories, it is anticipated the final report will document the agile development process and elements for each of the assessment categories will appear throughout the report.

3.1. Analysis of the problem (10 marks)	
3.1.1 Problem identification	a) Describe and justify the features that make the problem solvable by computational methods. b) Explain why the problem is amenable to a computational approach.
3.1.2 Stakeholders	a) Identify and describe those who will have an interest in the solution explaining how the solution is appropriate to their needs (this may be named individuals, groups or persona that describes the target end user).
3.1.3 Research the problem	a) Research the problem and solutions to similar problems to identify and justify suitable approaches to a solution. b) Describe the essential features of a computational solution explaining these choices. c) Explain the limitations of the proposed solution.
3.1.4 Specify the proposed solution	a) Specify and justify the solution requirements including hardware and software configuration (if appropriate). b) Identify and justify measurable success criteria for the proposed solution.

3.2. Design of the solution (15 marks)	
3.2.1 Decompose the problem	a) Break down the problem into smaller parts suitable for computational solutions justifying any decisions made.
3.2.2 Describe the solution	a) Explain and justify the structure of the solution. b) Describe the parts of the solution using algorithms justifying how these algorithms form a complete solution to the problem. c) Describe usability features to be included in the solution. d) Identify key variables / data structures / classes justifying choices and any necessary validation.
3.2.3 Describe the approach to testing	a) Identify the test data to be used during the iterative development and post development phases and justify the choice of this test data.

3.3. Developing the solution (30 marks)	
3.3.1 Iterative development process	a) Provide annotated evidence of each stage of the iterative development process justifying any decision made. b) Provide annotated evidence of prototype solutions justifying any decision made.
3.3.2 Development testing	a) Provide annotated evidence for testing at each stage justifying the reason for the test. b) Provide annotated evidence of any remedial actions taken justifying the decision made.
3.3.3 Post development testing	a) Provide annotated evidence of testing the solution for robustness at the end of the development process. b) Provide annotated evidence of usability testing.

3.4. Evaluation (15 marks)	
3.4.1 Success of the solution	a) Use the test evidence from the development and post development process to evaluate the solution against the success criteria from the analysis.
3.4.2 Describe the final product	a) Provide annotated evidence of the usability features from the design, commenting on their effectiveness.
3.4.3 Maintenance and development	a) Discuss the maintainability of the solution. b) Discuss potential further development of the solution.

2d. Prior learning and progression

Learners in England who are beginning an A level course are likely to have followed a Key Stage 4 programme of study. This course will enable learners to progress to higher study or to progress directly to employment.

There are no prior qualification requirements for this specification.

If learners wish to progress to higher education many universities now recognise the value of an A level qualification in Computer Science. Learners could then go on to a career in Medicine, Law, Business, Engineering or any of the Sciences.

It will provide learners with a range of transferable skills which will facilitate personal growth and foster cross curriculum links in areas such as maths, science and design and technology. Computer Science is a very creative subject and skills such as problem solving and analytical thinking will all be refined and explored as learners progress through the learning and assessment programme.

There are a number of Computer Science specifications at OCR. Find out more at www.ocr.org.uk

Draft

3 Assessment of OCR A Level in Computer Science

3a. Forms of assessment

This A Level in Computer Science is a linear qualification with 100% terminal external assessment.

This qualification consists of two examined components (01 and 02), externally assessed by OCR and one internally assessed and moderated non exam assessment component (03 or 04).

Both examinations are of 2 hours and 30 minutes duration, each with a 40% weighting. The non-exam assessment component weighted at 20%.

Computer systems (Component 01)

Learners answer **all** the questions. There will be a mix of questions including short answer, longer answer and some higher tariff questions that will test the quality of extended responses. Marks for these responses are integrated into the marking criteria.

The whole of the Computer systems content will be covered over the life of the specification.

Questions may contain for 'example'; following and correcting algorithms and programs, software development and legal and moral issues.

Algorithms and programming (Component 02)

Learners answer **all** the questions in Section A and **all** questions in Section B. There will be a mix of questions including short answer,

longer answer and some higher tariff questions that will test the quality of written communication via a level of response mark scheme.

The whole of the Algorithms and programming content will be covered over the life of the specification.

Section A will contain questions which may, for example, cover: writing algorithms and computational methods programming and programming techniques and problem solving. These questions may contain some shorter answer questions.

Section B will have a scenario set at the start of the section; this will contain information that will be used for the questions that follow. The questions will be largely of a higher tariff with problem solving algorithms and programming again forming the basis.

Programming project (Component 03 or 04)

The programming project will be submitted in the form of a report that will contain the solution to a problem, selected by the learner or centre, written in a suitable programming language. Within the report the learner must demonstrate their ability to analyse, design, develop, test and document using the principles learnt in computational thinking.

The report will be marked internally by the centre using the Programming project content in section 2c, in conjunction with the marking criteria in section 3e of this specification. It should then be sent to OCR for moderation.

3b. Assessment availability

There will be one examination series available each year in June to **all** learners.

This specification will be certificated from the June 2017 examination series onwards.

3c. Assessment objectives (AO)

There are three assessment objectives in OCR's A Level in Computer Science and these are detailed in the table below.

Learners are expected to demonstrate their ability to:

	Assessment Objectives	
AO1	Demonstrate knowledge and understanding of the principles and concepts of computer science, including abstraction, logic, algorithms and data representation	30–40%
AO2	Apply knowledge and understanding of the principles and concepts of computer science to analyse problems in computational terms	30–40%
AO3	Design, program and evaluate computer systems that solve problems, making reasoned judgements about these and presenting conclusions	30–40%

AO weightings in A Level Computer Science

The relationship between the assessment objectives and the components are shown in the following table:

Component	% of A Level Computer Science (H446)			
	AO1	AO2	AO3	Total
Computer systems (H446/01)	21	10	9	40
Algorithms and programming (H446/02)	10	24	6	40
Programming project (H446/03 or H446/04)	0	2	18	20
Total	31	36	33	100

3d. Assessment of extended responses

The assessment materials for this qualification provide learners with the opportunity to demonstrate their ability to

construct and develop a sustained line of reasoning and marks for extended responses are integrated into the marking criteria.

3e. Non exam assessment

Programming project (Component 03 or 04) Guidance

This qualification has one non exam assessment which takes the form of the Programming project (component 03 or 04). The project is a substantial piece of work which assesses a variety of different skills including the development and demonstration of computational thought processes. Whilst there are no specific controls linked to the assessment the following guidance should be considered before learners embark on this particular assessment.

Context Choice

Learners will need to choose in liaison with their teacher a well defined user driven problem. Whilst OCR will not be quality assuring the contexts chosen by learners the following criteria should be considered. The choice of project must enable learners to:

- meet all assessment requirements as contained in the specification
- use an appropriate programming language which is non-trivial and has a substantial coded element
- solve a problem sensibly within the constraints of resources available
- facilitate the successful completion of a whole task from its definition to its acceptance and evaluation by that user
- involve all elements of the skills of analysis, design, development and evaluation
- develop a stand alone or enhanced/modified solution.

Evidence Generation

It is important that learners establish thorough and robust working practices at an early stage. The projects should contain as standard the following:

Title	✓
Contents list	✓
Description of investigation	✓
Justification of investigation	✓
Analysis, design and methods used	✓
Evaluation	✓
Bibliography	✓
Pages numbered	✓

Appropriate annotated evidence should be used to support the report – e.g. screen dumps or photographs taken of screen layouts, for example. It is important that each learner generates their own individual, authentic evidence to show that they have met key assessment criteria. Any evidence submitted to demonstrate the development of solution must be able to be assessed without the use of any specific hardware or software. Whilst there is no word count for this particular assessment learners need to focus on the command words used within the assessment criteria. These command words indicate the depth of coverage that is required at each stage of the process and should drive the evidencing approach taken by learners.

The teacher has an important role in supporting learners through the evidence generation process. Whilst it is not appropriate for teachers to over direct learners in their generation of evidence it is acceptable for them to reinforce concepts and assist with the development of knowledge and understanding at various stages of the process. Acceptable forms of intervention include:

- offering learners advice about how best to approach each of the tasks
- exercising continuous supervision of work in order to monitor progress and to prevent plagiarism
- ensuring work is completed in accordance with the specification requirements and can be assessed in accordance with the specified marking criteria and procedures.

Assessing Evidence

It is important that centre staff responsible for marking learner projects are familiar with the principles of best fit assessment to ensure that a consistent marking approach is taken. For each of the marking criteria assessors select the band descriptor provided in the marking grid that most closely describes the quality of the work being marked. Marking should be positive, rewarding achievement rather than penalising failure or omissions.

The award of marks must be directly related to the marking criteria.

- Each band descriptor covers all of the relevant criteria for a particular section of the specification content related to analysis, design, development and evaluation.
- The descriptors for each section should be read and applied as a whole.
- An answer does not have to meet all of the requirements of a band descriptor before being placed in that band. It will be placed in a particular band when it meets more of the requirements of that band than it meets the requirements of other bands.

When deciding the mark within a band, the following criteria should be applied:

- the extent to which the statements within the band have been achieved.

For example:

- an answer that convincingly meets nearly all of the requirements of a band descriptor should be placed at or near the top of that band. Where the learner's work convincingly meets the statement, the highest mark should be awarded
- an answer that meets many of the requirements of the band descriptor should be placed in the middle of the band. Where the learner's work adequately meets the statement, the most appropriate mark in the middle range should be awarded
- if an answer is on the border-line between two bands but it is decided that it fits better the descriptors for the lower of these two bands, then it should be placed near the top of that band. Where the learner's work just meets the statement, the lowest mark should be awarded.

Assessors should use the full range of marks available to them and award full marks in any band of work that fully meets the descriptor. This is work that is 'the best one could expect from learners working at that level'.

Learners or centres must declare that the work is the learner's own (see section 4d).

Safeguarding Problems

It is important that centres put in place the necessary safeguards to both guard against and detect malpractice when it has occurred. As well as the use of the authentication process it is important that learners take responsibility for ensuring the work they produce is their own, is appropriately referenced and is a true reflection of their ability. OCR will take swift and decisive action if malpractice is detected and has not been dealt with by the centre

prior to authentication taking place. See www.ocr.org.uk

Ongoing Support

OCR will support teachers in the teaching and learning of this component by providing an extensive range of support material including sample assessment materials and schemes of work to provide exemplar ideas. These resources will be available online to support the launch of the qualification www.ocr.org.uk

Assessment criteria

Learners are expected to demonstrate their ability to analyse, design, develop, test, evaluate and document a program written in a suitable programming language. The problem must be solvable effectively in the chosen language so it is important there is a good match between the requirements of the problem and the language that is chosen.

The nature of the problem chosen is significant in that learners should be able to demonstrate, through their solutions, an appropriate range of skills identified by the specification; trivial problems, no matter how well they are executed will not be able to provide this evidence

Learners are expected to apply the computational thinking approach identified in Algorithms and programming (component 02)

to a practical coding problem and are expected to apply principles from an agile development approach to the project development.

The project should be assessed holistically and, while the assessment criterion are organised into discrete sections, it is unlikely evidence can be similarly organised in the learner's work. Evidence to support the assessment may be found throughout the project report and should be noted, by reference to page number, or otherwise, on the assessment grid. The assessment criterion are defined in mark bands and teachers should use a best-fit approach, selecting the most appropriate description and quality of match to that description, in order to place the work at the right level in the right mark band.

Programming project (Component 03 or 04) marking criteria

Analysis (maximum 10 marks)			
1–2 marks	3–5 marks	6–8 marks	9–10 marks
The candidate will have:			
- Identified some features that make the problem solvable by computational methods. - Identified suitable stakeholders for the project and described them and some of their requirements. - Identified some appropriate features to incorporate into their solution. - Identified some features of the proposed computational solution. - Identified some limitations of the proposed solution. - Identified some requirements for the solution. - Identified some success criteria for the proposed solution.	- Described the features that make the problem solvable by computational methods. - Identified suitable stakeholders for the project and described how they will make use of the proposed solution. - Researched the problem looking at existing solutions to similar problems identifying some appropriate features to incorporate into their solution. - Identified the essential features of the proposed computational solution. - Identified and described some limitations of the proposed solution. - Identified most requirements for the solution. - Identified some measurable success criteria for the proposed solution.	- Described the features that make the problem solvable by computational methods and why it is amenable to a computational approach. - Identified suitable stakeholders for the project and described them and how they will make use of the proposed solution and why it is appropriate to their needs. - Researched the problem in depth looking at existing solutions to similar problems identifying and describing suitable approaches based on this research. - Identified and described the essential features of the proposed computational solution. - Identified and explained any limitations of the proposed solution. - Specified the requirements for the solution including (as appropriate) any hardware and software requirements. - Identified measurable success criteria for the proposed solution.	- Described and justified the features that make the problem solvable by computational methods, explaining why it is amenable to a computational approach. - Identified suitable stakeholders for the project and described them explaining how they will make use of the proposed solution and why it is appropriate to their needs. - Researched the problem in depth looking at existing solutions to similar problems, identifying and justifying suitable approaches based on this research. - Identified the essential features of the proposed computational solution explaining these choices. - Identified and explained with justification any limitations of the proposed solution. - Specified and justified the requirements for the solution including (as appropriate) any hardware and software requirements. - Identified and justified measurable success criteria for the proposed solution.

0 marks = no response or no response worthy of credit.

Design (maximum 15 marks)			
1–4 marks	5–8 marks	9–12 marks	13–15 marks
The candidate will have:			
- Described elements of the solution using algorithms. - Described some usability features to be included in the solution. - Identified the key variables / data structures / classes (as appropriate to the proposed solution). - Identified some test data to be used during the iterative or post development phase of the process.	- Broken the problem down systematically into a series of smaller problems suitable for computational solutions describing the process. - Defined the structure of the solution to be developed. - Described the solution fully using appropriate and accurate algorithms. - Described the usability features to be included in the solution. - Identified the key variables / data structures / classes (as appropriate to the proposed solution) and any necessary validation. - Identified the test data to be used during the iterative development of the solution. - Identified any further data to be used in the post development phase.	- Broken the problem down systematically into a series of smaller problems suitable for computational solutions explaining the process. - Defined in detail the structure of the solution to be developed. - Described the solution fully using appropriate and accurate algorithms explaining how these algorithms form a complete solution to the problem. - Described, explaining choices made, the usability features to be included in the solution. - Identified and justified the key variables / data structures / classes (as appropriate to the proposed solution) explaining any necessary validation. - Identified and justified the test data to be used during the iterative development of the solution. - Identified and justified any further data to be used in the post development phase.	- Broken the problem down systematically into a series of smaller problems suitable for computational solutions, explaining and justifying the process. - Defined in detail the structure of the solution to be developed. - Described the solution fully using appropriate and accurate algorithms justifying how these algorithms form a complete solution to the problem. - Described, justifying choices made, the usability features to be included in the solution. - Identified and justified the key variables / data structures / classes (as appropriate to the proposed solution) justifying and explaining any necessary validation. - Identified and justified the test data to be used during the iterative development of the solution. - Identified and justified any further data to be used in the post development phase.

0 marks = no response or no response worthy of credit.

Developing the solution (maximum 30 marks)			
Iterative development (maximum 15 marks)			
1–4 marks	5–8 marks	9–12 marks	13–15 marks
The candidate will have:			
• Provided evidence of some iterative development.	• Provided evidence for most stages of the iterative development process describing what they did at each stage.	• Provided evidence of each stage of the iterative development process relating this to the break down of the problem from the analysis stage and explaining what they did at each stage. • Provided evidence of some prototype versions of their solution.	• Provided evidence of each stage of the iterative development process relating this to the break down of the problem from the analysis stage and explaining what they did and justifying why. • Provided evidence of prototype versions of their solution for each stage of the process.
Development testing (maximum 10 marks)			
1–2 marks	3–5 marks	6–8 marks	9–10 marks
The candidate will have:			
• Provided some evidence of testing during the iterative development process.	• Provided some evidence of testing during the iterative development process. • Provided evidence of some failed tests and the remedial actions taken.	• Provided evidence of testing at most stages of the iterative development process. • Provided evidence of some failed tests and the remedial actions taken with some explanation of the actions taken.	• Provided evidence of testing at each stage of the iterative development process. • Provided evidence of any failed tests and the remedial actions taken with full justification for any actions taken.
Post-development testing (maximum 5 marks)			
1 mark	2 marks	3–4 marks	5 marks
The candidate will have:			
• Provided evidence of some post development testing.	• Provided evidence of final product testing for function.	• Provided annotated evidence of post development testing for function. • Provided annotated evidence for usability testing.	• Provided annotated evidence of post development testing for function and robustness. • Provided annotated evidence for usability testing.

0 marks = no response or no response worthy of credit.

Evaluation (maximum 15 marks)			
1–4 marks	5–8 marks	9–12 marks	13–15 marks
The candidate will have:			
- Commented on the success or failure of the solution with some reference to test data. - The information is basic and communicated in an unstructured way. The information is supported by limited evidence and the relationship to the evidence may not be clear.	- Cross referenced some of the test evidence with the success criteria and commented on the success or otherwise of the solution. - Provided evidence of usability features. - Identified some limitations on the solution. - The information has some relevance and is presented with limited structure. The information is supported by limited evidence.	- Used the test evidence to cross reference with the success criteria to evaluate the solution identifying whether the criteria have been met, partially met or unmet. - Provided comments on how any partially or not met criteria could be addressed in further development. - Provided evidence of the usability features. - Considered maintenance issues and limitations of the solution. - There is a line of reasoning presented with some structure. The information presented is in the most part relevant and supported by some evidence.	- Used the test evidence to cross reference with the success criteria to evaluate the solution explain how the evidence shows that the criteria has been fully, partially or not met in each case. - Provided comments on how any partially or unmet criteria could be addressed in further development. - Provided evidence of the usability features justifying their success, partial success or failure as effective usability features. - Provided comments on how any issues with partially or unmet usability features could be addressed in further development. - Considered maintenance issues and limitations of the solution. - Described how the program could be developed to deal with limitations of potential improvements / changes. - There is a well developed line of reasoning which is clear and logically structured. The information presented is relevant and substantiated.

0 marks = no response or no response worthy of credit.

3f. Synoptic assessment

The three components encourage learners to develop their knowledge, understanding and skills in different aspects of Computer Science.

The different assessments encourage learners to demonstrate that they can draw on knowledge and understanding from other components to structure their responses.

The Programming project (component 03 or 04) is truly synoptic in its approach to assessment. Through the context that learners select they will apply the knowledge, understanding and skills that they have developed throughout the A level course.

The emphasis of synoptic assessment is to encourage the understanding of Computer Science as a discipline.

It is envisaged that the Computer systems (component 01) would be taught first but there are elements of the Algorithms and programming (component 02) that should also be considered as Computer systems are taught in order that learners are fully aware of the way problem solving through computational thinking is achieved.

The Algorithms and programming component (02) builds on the knowledge and understanding gained in Computer systems (01). Algorithms and programming will contain synoptic assessment.

The Programming project (component 03 or 04) draws on all the elements of the other components in order to enable learners to develop synoptic thought processes in their approach to the solving of a real life scenario. As part of this process, learners will draw on knowledge, understanding and skills that have been embedded in components 01 and 02.

3g. Calculating qualification results

A learner's overall qualification grade for an A Level in Computer Science will be calculated by adding together their marks from the three components taken to give their total weighted mark.

This mark will then be compared to the qualification level grade boundaries for the entry option taken by the learner and for the relevant exam series to determine the learner's overall qualification grade.

4 Admin: what you need to know

The information in this section is designed to give an overview of the processes involved in administering this qualification so that you can speak to your exams officer. All of the following processes require you to submit something to OCR by a specific deadline.

More information about these processes, together with the deadlines, can be found in the *OCR Admin Guide and Entry Codes: 14–19 Qualifications*, which can be downloaded from the OCR website: www.ocr.org.uk

4a. Pre-assessment

Estimated entries

Estimated entries are your best projection of the number of learners who will be entered for a qualification in a particular series.

Estimated entries should be submitted to OCR by the specified deadline. They are free and do not commit your centre in any way.

Final entries

Final entries provide OCR with detailed data for each learner, showing each assessment to be taken. It is essential that you use the correct entry code, considering the relevant entry rules and ensuring that you choose the entry option for the moderation method you intend to use.

Final entries must be submitted to OCR by the published deadlines or late entry fees will apply.

All learners taking A Level in Computer Science must be entered for one of the entry options on the following table:

Entry option		Components		
Entry code	Title	Code	Title	Assessment type
H446 A	Computer Science (OCR Repository)	01	Computer systems	External Assessment
		02	Algorithms and programming	External Assessment
		03	Practical project (OCR Repository)	Non exam assessment (OCR Repository)
H446 B	Computer Science (Postal moderation)	01	Computer systems	External Assessment
		02	Algorithms and programming	External Assessment
		04	Programming project (Postal moderation)	Non exam assessment (Postal moderation)
H446 C*	Computer Science Non exam assessment (carried forward)	01	Computer systems	External Assessment
		02	Algorithms and programming	External Assessment
		80	Programming project (carried forward)	Non exam assessment (carried forward)

*Entry option H446 C should only be selected for learners who are retaking the qualification and who want to carry forward their mark for the non exam assessment.

Estimated grades

An estimated grade is the grade the centre expects a learner to achieve for a qualification. These should be submitted to OCR by the specified deadline

4b. Accessibility and special consideration

The A level qualification and subject criteria have been reviewed in order to identify any feature which could disadvantage candidates who share a protected characteristic as defined by the Equality Act 2010. All reasonable steps have been taken to minimise any such disadvantage.

Reasonable adjustments and access arrangements allow learners with special educational needs, disabilities or temporary injuries to access the assessment and show what they know and can do, without changing the demands of the assessment. Applications for these should be made before the

examination series. Detailed information about eligibility for access arrangements can be found in the *JCQ Access Arrangements and Reasonable Adjustments*.

Special consideration is a post-assessment adjustment to marks or grades to reflect temporary injury, illness or other indisposition at the time the assessment was taken.

Detailed information about eligibility for special consideration can be found in the *JCQ A guide to the special consideration process*.

4c. External assessment arrangements

Regulations governing examination arrangements are contained in the *JCQ Instructions for conducting examinations*.

4d. Non exam assessment

Regulations governing arrangements for internal assessments are contained in the *JCQ Instructions for conducting coursework*.

Authentication of learners' work

Learners and centres must declare that the work is the learner's own.

Teachers must declare that the work submitted for internal assessment is the

learner's own work by submitting a centre authentication form (CCS160) for each internally assessed component. This should be sent to the moderator at the same time as the mark.

Internal standardisation

Centres must carry out internal standardisation to ensure that marks awarded by different teachers are accurate and consistent across all learners entered for the component from that centre.

Moderation

The purpose of moderation is to bring the marking of internally-assessed components in all participating centres to an agreed standard. This is achieved by checking a sample of each centre's marking of learners' work.

Following internal standardisation, centres submit marks to OCR and the moderator. If there are fewer than 10 learners, all the work should be submitted for moderation at the same time as marks are submitted.

Once marks have been submitted to OCR and your moderator, centres will receive a moderation sample request. Samples will include work from across the range of attainment of the learners' work.

There are two ways to submit a sample:

Moderation via the OCR Repository – Where you upload electronic copies of the work included in the sample to the OCR Repository and your moderator accesses the work from there.

Postal moderation – Where you post the sample of work to the moderator.

The method that will be used to submit the moderation sample must be specified when making entries. The relevant entry codes are given in Section 4a

All learners work must be submitted using the same entry option. It is not possible for centres to offer both options within the same series.

Centres will receive the outcome of moderation when the provisional results are issued. This will include:

Moderation Adjustments Report – Listing any scaling that has been applied to internally-assessed components.

Moderator Report to Centres – A brief report by the moderator on the internal assessment of learners' work.

4e. Results and certificates

Grade Scale

A level qualifications are graded on the scale: A*, A, B, C, D, E, where A* is the highest. Learners who fail to reach the minimum

standard for E will be Unclassified (U). Only subjects in which grades A* to E are attained will be recorded on certificates.

Results

Results are released to centres and learners for information and allow any queries to be resolved **before** certificates are issued.

Centres will have access to the following results information for each learner:

- the grade for the qualification
- the raw mark for each component
- the total weighted mark for the qualification.

The following supporting information will be available:

- raw mark grade boundaries for each component
- weighted mark grade boundaries for the qualification.

Until certificates are issued, results are deemed to be provisional and may be subject to amendment. A learner's final results will be recorded on an OCR certificate.

The qualification title will be shown on the certificate as 'OCR Level 3 Advanced GCE in Computer Science'.

4f. Post-results services

A number of post-results services are available:

- **Enquiries about results** – If you are not happy with the outcome of a learner's results, centres may submit an enquiry about results.
- **Missing and incomplete results** – This service should be used if an individual subject result for a learner is missing, or the learner has been omitted entirely from the results supplied.
- **Access to scripts** – Centres can request access to marked scripts.

4g. Malpractice

Any breach of the regulation for the conduct of examinations and coursework may constitute malpractice (which includes maladministration) and must be reported to OCR as soon as it is detected.

Detailed information on malpractice can be found in the JCQ *Suspected Malpractice in Examinations and Assessments: Policies and Procedures*.

5 Appendices

5a. Grade descriptors

Details to be confirmed by Ofqual.

5b. Overlap with other qualifications

The knowledge, understanding and skills that are developed throughout this qualification are distinct and have very little overlap with other qualifications. This overlap may occur

only at level 5 in the hardware and software elements of ICT Cambridge Technicals and GCE Applied ICT.

5c. Mathematical skills

All AS level and A level Computer Science qualifications must contain a minimum of 10% mathematical skills.

5d. Languages and logic

The tables below show languages and logic that will be used in the external assessments and indicates the limits and scope of each.

Centres are free to go beyond these parameters.

Pseudocode

The following guide shows the format pseudocode will appear in the examined components. It is provided to allow you to give learners familiarity before the exam. Learners are not expected to memorise the syntax of this pseudocode and when asked may provide answers in any style of pseudocode they choose providing its meaning could be reasonably inferred by a competent programmer.

Outputting to Screen

`print (string)`

Example

```
print("hello")
```

Taking Input from User

`variable=input (prompt to user)`

Example

```
name=input("Please enter your name")
```

Iteration – Count Controlled

```
for i=0 to 7
    print("Hello")
next i
```

Will print hello 8 times (0-7 inclusive).

Iteration – Condition Controlled

```
while answer!="computer"
    answer=input("What is the password?")
endwhile

do
    answer=input("What is the password?")
until answer=="computer"
```

Logical Operators

AND OR NOT

e.g.

```
while x<=5 AND flag==false
```

Comparison Operators

<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code><</code>	Less than
<code><=</code>	Less than or equal to
<code>></code>	Greater than
<code>>=</code>	Greater than or equal to

Arithmetic Operators

<code>+</code>	Addition e.g. $x=6+5$ gives 11
<code>-</code>	Subtraction e.g. $x=6-5$ gives 1
<code>*</code>	Multiplication e.g. $x=12*2$ gives 24
<code>/</code>	Division e.g. $x=12/2$ gives 6
<code>MOD</code>	Modulus e.g. $12\text{MOD}5$ gives 2
<code>DIV</code>	Quotient e.g. $17\text{DIV}5$ gives 3
<code>^</code>	Exponentiation e.g. 3^4 gives 81

Selection

Selection will be carried out with if/else and switch/case

if/else

```
if entry=="a" then
    print("You selected A")
elseif entry=="b" then
    print("You selected B")
else
    print("Unrecognised selection")
endif
```

switch/case

```
switch entry:
    case "A":
        print("You selected A")
    case "B":1
        print("You selected B")
    default:
        print("Unrecognised selection")

endswitch
```

String Handling

To get the length of a string:
stringname.length

To get a substring:

```
stringname.substring(startingPosition, numberOfCharacters)
```

NB The string will start with the 0th character.

Example

```
someText="Computer Science"

print(someText.length)
print(someText.substring(3,3))
```

Will display

```
16
put
```

Subroutines

```
function triple(number)
    return number*3
endfunction
```

Called from main program

```
y=triple(7)
```

```
procedure greeting(name)
    print("hello"+name)
endprocedure
```

Called from main program

```
greeting("Hamish")
```

Unless stated values passed to subroutines can be assumed to be passed by value. If this is relevant to the question byVal and byRef will be used. In the case below x is passed by value and y is passed by reference.

```
procedure foobar(x:byVal, y:byRef)
    ...
    ...
endprocedure
```

Arrays

Arrays will be 0 based and declared with the keyword *array*.

```
array names[5]
names[0]="Ahmad"
names[1]="Ben"
names[2]="Catherine"
names[3]="Dana"
names[4]="Elijah"

print(names[3])
```

Example of 2D array:

```
Array board[8,8]
board[0,0]="rook"
```

Reading to and Writing from Files

To open a file to read from `openRead` is used and `readLine` to return a line of text from the file.

The following program makes `x` the first line of `sample.txt`

```
myFile = openRead("sample.txt")
x = myFile.readLine()
myFile.close()
```

`endOfFile()` is used to determine the end of the file. The following program will print out the contents of `sample.txt`

```
myFile = openRead("sample.txt")
while NOT myFile.endOfFile()
    print(myFile.readLine())
endwhile
myFile.close()
```

To open a file to write to `openWrite` is used and `writeLine` to add a line of text to the file. In the program below `hello world` is made the contents of `sample.txt` (any previous contents are overwritten).

```
myFile = openWrite("sample.txt")
myFile.writeLine("Hello World")
myFile.close()
```

Comments

Comments are denoted by `//`

```
print("Hello World") //This is a comment
```

Object-Oriented

Object oriented code will match the pseudocode listed above with the following extensions:

Methods and Attributes:

Methods and attributes can be assumed to be public unless otherwise stated. Where the access level is relevant to the question it will always be explicit in the code denoted by the keywords.

`public` and `private`.

```
private attempts = 3
```

```
public procedure setAttempts(number)
    attempts=number
endprocedure
```

```
private function getAttempts()
    return attempts
endfunction
```

Methods will always be instance methods, learners aren't expected to be aware of static methods. They will be called using `object.method` so

```
player.setAttempts(5)

print(player.getAttempts())
```

Constructors and Inheritance

Inheritance is denoted by the `inherits` keyword, superclass methods will be called with the keyword `super`. i.e. `super.methodName(parameters)` in the case of the constructor this would be `super.new()`

Constructors will be procedures with the name `new`.

```
class Pet

  private name
  public procedure new(givenName)
    name=givenName

  endprocedure

endclass

class Dog inherits Pet

  private breed

  public procedure new(givenName, givenBreed)
    super.new(givenName)
    breed=givenBreed
  endprocedure

endclass
```

HTML

Learners are expected to have an awareness of the following tags. Any other tags used will be introduced in the question.

```
<html>
<link> to link to a CSS file
<head>

<title>

<body>

<h1> <h2> <h3>

<img> including the src, alt, height and width attributes.

<a> including the href attribute.

<div>
```

<form>

<input> where the input is a textbox (i.e. has the attribute `type="text"` and another attribute `name` to identify it) or a submit button (i.e. has the attribute `type="submit"`)

<p>

<script>

Any other elements used will be explained in the question.

CSS

Learners are expected to be able to use CSS directly inside elements using the style attribute

```
<h1 style="color:blue;">
```

and external style sheets. In the style sheets they should be able to use CSS to define the styling of elements:

```
h1{  
    color:blue;  
}
```

classes

```
.infoBox{  
    background-color: green;  
}
```

and Identifiers

```
#menu{  
    background-color: #A2441B  
}
```

They are expected to be familiar with the following properties.

```
background-color  
border-color  
border-style  
border-width  
color with named and hex colours  
font-family  
font-size  
height  
width
```

Any other properties used will be explained in the question.

JavaScript

Learners are expected to be able to follow and write basic JavaScript code. It is hoped they will get practical experience of JavaScript in their study of the course. They will not be expected to commit exact details of syntax to memory. Questions in the exam will not penalise learners for minor inaccuracies in syntax. Learners *will* be expected to be familiar with the JavaScript equivalents of the structures listed in the pseudocode section (with the exception of input and output (see below). They will not be expected to use JavaScript for Object Oriented programming or file handling. Questions will not be asked in JavaScript where something is passed to a subroutine by value or reference is relevant.

Input

Input will be taken in by reading values from a form. *NB learners will not be expected to memorise the method for doing this as focus will be on what they do with that input once it is received.*

Output

By changing the contents of an HTML element

```
chosenElement = document.getElementById("example");  
chosenElement.innerHTML = "Hello World";
```

By writing directly to the document

```
document.write("Hello World");
```

By using an alert box

```
alert("Hello World");
```

Any other JavaScript used will be explained in the question.

Little Man Computer Instruction Set

In questions mnemonics will always be given according to the left hand column below. Different implementations of LMC have slight variations in mnemonics used and to take this into account the alternative mnemonics in the right hand column will be accepted in learners' answers.

| Mnemonic | Instruction | Alternative mnemonics accepted |
|----------|--------------------|--------------------------------|
| ADD | Add | |
| SUB | Subtract | |
| STA | Store | STO |
| LDA | Load | LOAD |
| BRA | Branch always | BR |
| BRZ | Branch if zero | BZ |
| BRP | Branch if positive | BP |
| INP | Input | IN, INPUT |
| OUT | Output | |
| HLT | End program | COB, END |
| DAT | Data location | |

Structured Query Language (SQL)

Learners will be expected to be familiar with the structures below. Should any other aspects of SQL be used they will be introduced and explained in the question.

SELECT *(including nested SELECTs)*

FROM *(including use of * wildcard)*

WHERE

LIKE *(with % used as a wildcard)*

AND

OR

DELETE

INSERT

DROP

JOIN *(Which is equivalent to INNER JOIN, there is no expectation to know about outer, left and right joins)*

Boolean Algebra

When Boolean algebra is used in questions the notation described below will be used. Other forms of notation exist and below are also a list of accepted notation we will accept from learners.

Conjunction

Notation used

$\wedge$ e.g. $A \wedge B$

| A | B | $A \wedge B$ |
|---|---|--------------|
| T | T | T |
| T | F | F |
| F | T | F |
| F | F | F |

Alternatives accepted:

AND e.g. A AND B
e.g. A.B

Disjunction

Notation used:

$\vee$ e.g. $A \vee B$

| A | B | $A \vee B$ |
|---|---|------------|
| T | T | T |
| T | F | T |
| F | T | T |
| F | F | F |

Alternatives accepted:

OR e.g. A OR B

+ e.g. A+B

Negation

Notation used:

$\neg$ e.g. $\neg A$

| A | $\neg A$ |
|---|----------|
| T | F |
| F | T |

Alternatives Accepted:

bar e.g. $\overline{A}$

$\sim$ e.g. $\sim A$

NOT e.g. NOT A

Exclusive Disjunction

Notation used:

$\veebar$ e.g. $A \veebar B$

| A | B | $A \veebar B$ |
|---|---|---------------|
| T | T | F |
| T | F | T |
| F | T | T |
| F | F | F |

Alternatives accepted:

XOR e.g. A XOR B

$\oplus$ e.g. $A \oplus B$

Equivalence / Iff

Notation used:

$\equiv$ e.g. $(A \wedge B) \equiv \neg(\neg A \vee \neg B)$

Alternatives accepted:

$\leftrightarrow$